

Síntesis, traducción y verificación de sistemas de ecuaciones utilizados en Protocolos de Conocimiento Cero

Lucía Martínez Martínez

Grado en Ingeniería Informática
Universidad Complutense de Madrid



Trabajo de Fin de Grado
Curso 2025 - 2026

Directores:
Clara Rodríguez Núñez
Alejandro Hernández Cerezo

Índice general

1. Introducción a Circom	3
1.1. Zero Knowledge Proof	3
1.2. CIRCOM	4
1.3. R1CS	6
1.4. Snarkjs	7
2. Descripción del problema	8
2.1. Ejemplo del problema	8
2.2. Reglas de transformación	8
2.3. Ejemplo con constraints generadas automáticamente	8
3. Implementación	9
3.1. Explicación y ejemplo de la construcción de las reglas	9
4. Experimentos	10
4.1. Aplicación de la generación automática sobre juegos simples	10
4.2. Aplicación de la generación automática sobre circomLib y comparación de constraints	10
4.3. Caso de uso, verificación y búsqueda de bugs	10

Introducción a Circom

1.1. Zero Knowledge Proof

En términos criptográficos, una prueba de conocimiento cero, conocida como Zero Knowledge Proof (o sus siglas inglesas ZKP), es una **familia de protocolos** que permite el establecimiento de métodos cuyo objetivo es permitir que una parte (**prover**) demuestre a otra (**verifier**) la validez de una afirmación **sin revelar información sobre la misma**, más allá de su validez.

Alejandro (prover) tiene el décimo de la lotería de Navidad que ha sido premiado y quiere demostrarle a **Clara (verifier)** que posee el número ganador, pero sin que ésta sea conocedora del mismo (y así evitar que pueda cobrar el premio o se lo robe). Para ello, utilizan una caja fuerte que solo se abre introduciendo la combinación del decimo premiado.

Clara escribe en un papel un dígito aleatorio, lo introduce en la caja y la cierra. Posteriormente, Clara se gira, de forma que no ve nada más de la caja.

Alejandro, que conoce el número del décimo, introduce la combinación, abriendo así la caja y leyendo el número escrito por Clara en el papel en voz alta. En ese momento, Clara sabe que ha podido abrir la caja, y por tanto, obtiene una prueba de que Alejandro es conocedor de esa información.

Sin embargo, Clara podría sospechar que Alejandro ha adivinado el dígito al azar, ya que la probabilidad es del 10 %, por lo que Clara decide repetir la prueba varias veces.

No obstante, si observamos $0,1 * 0,1 = 0,01$, se reduce la probabilidad inicial de 0,1 a 0,01. Es decir, a medida que las repeticiones aumentan, menor es la probabilidad de que se acierte al azar, ya que ésta converge a 0, alcanzando así una certeza práctica sin haber revelado ni un solo dígito del décimo.

Ejemplo 1.1: Zero Knowledge Proof

Con hemos visto en el *Ejemplo 1.1*, el concepto de **ZKP** queda ilustrado de una forma muy intuitiva. Sin embargo, para trasladar esta lógica al contexto criptográfico, es necesario formalizar estas interacciones mediante protocolos específicos.

Históricamente, el término de ZKP apareció en los años 80 como un concepto puramente teórico, hasta su desarrollo en la práctica. Dicha noción surgió con la intención de reforzar tanto la seguridad como la privacidad en el ámbito criptográfico.

La **criptografía** constituye el pilar fundamental de la seguridad en el mundo digital, permitiendo así proteger información sensible frente a amenazas. Las técnicas criptográficas nos ofrecen la capacidad de garantizar confidencialidad así como un equilibrio entre transparencia y privacidad.

Gracias a este equilibrio, las técnicas criptográficas pudieron llevar a la práctica el concepto teórico ZKP en protocolos prácticos. Actualmente, esto ha llevado a la creación de grandes familias de protocolos, destacando especialmente **zk-SNARKs**.

Succinct Non-Interactive Argument of Knowledge, mayormente conocido por sus siglas **zk-SNARKs**, destacan por su reducido y concreto tamaño de prueba y corto tiempo de verificación. Sin embargo, presentan la desventaja de requerir una fase inicial conocida como **configuración de confianza**.

Una configuración de confianza es una fase en la que se produce la generación de valores aleatorios que deben destruirse de forma inmediata. Si estos valores aleatorios son expuestos, se compromete la seguridad del sistema.

El **motivo** de que estos número aleatorios deban borrarse inmediatamente es debido a que cualquier entidad conocedora de esos valores podría almacenarlos y así generar **pruebas falsas**, de forma que se podría vulnerar la seguridad del sistema.

Dada la complejidad que implica implementar estos protocolos desde cero, se han desarrollado lenguajes de programación específicos denominados **Domain Specific Languages**, conocidos como DSL.

El objetivo es que los desarrolladores que no son expertos en criptografía puedan programar las declaraciones que desean demostrar. Es en este contexto donde cobra importancia **Circom**, una herramienta creada con el fin de facilitar la creación de circuitos y su posterior transformación en sistemas de restricciones.

1.2. CIRCOM

Circom es un lenguaje de programación basado en la descripción de circuitos (DSL), cuyo objetivo principal no es el cálculo, sino definir un sistema en el que las **señales** se relacionan entre sí.

Una señal es un componente que actúa como cable dentro del circuito, el cual transporta valores a través de las puertas lógicas del mismo y que forma parte de las ecuaciones que el programador desea verificar. Estas señales constituyen las restricciones que formarán parte del archivo **R1CS**.

Esta visión es fundamental, ya que las ZKP no pueden interpretar código directamente; requieren que la computación se descomponga en una estructura de restricciones algebraicas denominada R1CS, que detallaremos en **1.3. R1CS**.

Además de su finalidad criptográfica, es importante entender la diferencia entre un lenguaje basado en circuitos a uno de propósito general, siendo Circom basado en circuitos donde la estructura del mismo debe ser estática; esto implica que cualquier valor determinante en la estructura del circuito (como tamaños de arrays, número de iteraciones de un bucle, etc..) **debe conocerse en tiempo de compilación**. Esto se debe a que así aseguramos que el sistema de restricciones R1CS que se genera en base a este circuito, sea fijo.

El proceso de compilación de Circom refleja un paradigma híbrido entre lo **declarativo e imperativo**. De esta forma, el compilador genera dos archivos esenciales para generar la prueba:

- **Archivo R1CS:** Contiene el conjunto de todas las restricciones algebraicas que definen la lógica del circuito.
- **Programa de Testigo (Witness):** Un ejecutable (en formato C++ o WASM) encargado de calcular los valores de todas las señales del circuito, incluyendo los valores intermedios, a partir de las entradas proporcionadas.

Partiendo de estos dos archivos, podemos generar la prueba por medio de **Snarkjs**. Esta herramienta toma el archivo R1CS y el Witness obtenidos tras la compilación de un programa Circom para generar la prueba asociada. Analizaremos en detalle su funcionamiento y comandos específicos en la sección **1.4. Snarkjs**.

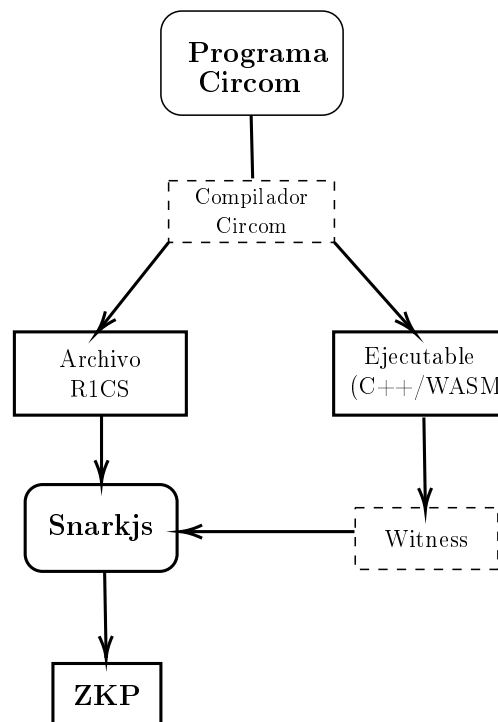


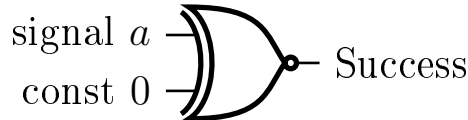
Figura 1.1: Proceso de transformación a ZKP desde Circom

Como hemos mencionado anteriormente, las señales son las restricciones establecidas por el programador las cuales quiere verificar. A continuación un ejemplo de un programa Circom.

```

1      template var_zero(){
2          signal input a;
3          a * 1 === 0;
4      }
5
6      main component = var_zero();

```



Ejemplo 1.2: Archivo var_zero.circom

1.3. R1CS

Rank-1 Constraint System (conocido como **R1CS**), es un sistema de representación matemático diseñado para transformar tareas computacionales complejas en un conjunto de restricciones algebraicas. Su relevancia incide en la propiedad de que generar la solución a un problema puede ser caro, mientras que su **verificación es muy eficiente y rápida**, clave en las pruebas de conocimiento cero.

R1CS impone restricciones matemáticas complejas. Las ecuaciones, aparte de no poder exceder el segundo grado (*condición necesaria, pero no suficiente*), cada restricción debe poder expresarse como el producto de dos combinaciones lineales de variables, teniendo como resultado una tercera combinación lineal:

$$A \cdot B - C = 0$$

Donde A , B y C son combinaciones lineales de las señales del circuito. En términos prácticos, esto implica que una sola restricción solo puede contener una multiplicación entre variables. Operaciones que incluyan múltiples productos independientes, aunque sean de segundo grado, deben ser descompuestas en varias restricciones consecutivas por medio de variables auxiliares con el fin de cumplir con este estándar.

Siguiendo el *Ejemplo 1.2*, la declaración que utilizamos para añadir la restricción $a == 0$ a la hora de compilarse se transformaría en formato R1CS.

De esta forma, la restricción $a * 1 === 0$ se traduce al formato R1CS como el producto de dos combinaciones lineales A y B tal que $A \times B = C$, donde $A=a$, $B=1$ y $C=0$, forzando así que el valor de la señal a sea nulo.

Dada la declaración:

$$(a) \times (1) = (0)$$

Donde la estructura $A \cdot B = C$ se satisface asignando:

- $A = a$ (Combinación lineal 1)
- $B = 1$ (Combinación lineal 2)
- $C = 0$ (Resultado de la restricción)

Ejemplo 1.3: Transformación de `var_zero.circom` a formato R1CS

Gracias a R1CS, es posible que el código **escrito en Circom pueda transformarse en un sistema de restricciones**, consiguiendo así que la verificación sea sumamente eficiente.

1.4. Snarkjs

Descripción del problema

- 2.1. Ejemplo del problema
- 2.2. Reglas de transformación
- 2.3. Ejemplo con constraints generadas automáticamente

Implementación

3.1. Explicación y ejemplo de la construcción de las reglas

Experimentos

- 4.1. Aplicación de la generación automática sobre juegos simples
- 4.2. Aplicación de la generación automática sobre circomLib y comparación de constraints
- 4.3. Caso de uso, verificación y búsqueda de bugs